

Logic

Convey "not" by `!`, "and" by `&&`, "or" by `||`.

For Loop vs While Loop

- If know how many times should loop, choose `for(initialize; condition; step)`.
- If know the collection wanted to enumerate, choose `for(T element: collection)`.
- Else choose `while(condition)`.

ArrayList

A kind of list, mostly used. Created by `ArrayList<T> a = new ArrayList<>()`. Containing the methods: `a.add(var)`, `a.get(index)`, `a.size()`, `a.remove(index)`, `a.contains(var)`.

HashSet

Just a set, no duplicate. Created by `HashSet<T> s = new HashSet<>()`. Containing the methods: `s.add(var)`, `s.remove(var)`, `s.contains(var)`, `s.size()`.

Stack

First in last out(FILO), used in DFS. Created by `Stack<T> s = new Stack<>()`. Containing the the methods: `s.pop()`, `s.push(var)`, `s.isEmpty()`, `s.peek()`.

Queue

First in first out(FIFO), used in BFS. Created by `Queue<T> q = new LinkedList<>()`. Containing the methods: `q.poll()`, `q.add(var)`, `q.isEmpty()`, `s.peek()`.

Map

Key-value pairs. Key must be unique. Created by `Map<K, V> m = new Map<>()`. Containing the methods: `m.put(k, v)`, `m.get(k)`, `m.remove(k)`, `m.containsKey(k)`, `m.keySet()`.

Recursion

Consisting of two case: base case, recursion case.

```
static ArrayList<String> allWords(int n, ArrayList<Character> alphabet){
    if (n == 0) return new ArrayList<>();
    ArrayList<String> pre = allWords(n-1, alphabet);
    ArrayList<String> res = new ArrayList<>();
    for (String s : pre){
        for (Character c : alphabet ) res.add( s + c );
    }
    return res ;
}
```

```

static ArrayList<String> allSubstrings(String s) {
    ArrayList<String> result = new ArrayList<String>();
    if (s.length() == 0) result.add("");
    else {
        ArrayList<String> rest = allSubstrings(s.substring(1));
        result.addAll(rest);
        for (String str : rest) result.add(s.charAt(0) + str);
    }
    return result;
}

static ArrayList<String> allBinaryStrings(int n) {
    ArrayList<String> result = new ArrayList<String>();
    if (n == 0) result.add("");
    else {
        ArrayList<String> rest = allBinaryStrings(n - 1);
        for (String s : rest) {
            result.add("0" + s);
            result.add("1" + s);
        }
    }
    return result;
}

```

Linked List

Consisting by two class: `List<T>` and `Node<T>`. Get the value of the node by `p.value`.

- Enumerate

To the last node: `while (p != null) p = p.next;`. `p` is null after the last node after the loop.

To the First Node: `while (p != null) p = p.prev;`. `p` is null before the first node after the loop.

- Add Node

```

void addAtFirst(T x){
    Node p = this.first;
    Node newNode = new Node<>();
    newNode.value = x;
    newNode.next = p;
    p.prev = newNode;
}

void addAtLast(T x){
    Node p = this.first;
    while(p.next != null) p = p.next; // Traverse to last node
    Node newNode = new Node<>();
    newNode.value = x;
    newNode.prev = p;
    p.next = newNode;
}

```

```

}
void insertAt(int i, T x){
    Node p = this.first;
    for(int j = 0; j < i-1; j ++){ p = p.next;
    Node newNode = new Node<>();
    newNode.value = x;
    // inserting
    newNode.prev = p;
    newNode.next = p.next;
    p.next = newNode;
    newNode.next.prev = newNode;
}

```

- Remove Node

```

void removeLast(Node p) {
    while (p.next != null) p = p.next;
    // Remove last node
    if (p.prev != null) p.prev.next = null;
    p.prev = null;
}
void removeFirst(Node p){
    while (p.prev != null) p = p.prev;
    if (p.next != null) p.next.prev = null;
    p.next = null;
}
void removeAt(int i){
    Node p = this.first;
    for (int j = 0; j < i; j ++){ p = p.next;
    p.prev.next = p.next;
    p.next.prev = p.prev;
    p.prev = null;
    p.next = null;
}
}

```

- Reverse

If it is a method in a class, replace `list` to `this`.

```

static <T> List<T> reversed(List<T> list){
    List<T> ans = new List<>();
    for (Node p = list.first; p != null; p = p.next){
        Node newNode = new Node<>();
        newNode.value = p.value;
        newNode.next = ans.first;
        ans.first = newNode;
    }
    return ans;
}

```

```

void reverse(List<T> list){
    Node pre = null;
    Node p = list.first;
    while (p != null) {
        Node nxt = p.next;
        p.next = pre;
        pre = p;
        p = nxt;
    }
    list.first = pre;
}

```

Tree

If you see **tree**, recall recursive DFS first, then iterative DFS, BFS at last.

- Recursive DFS

```

void dfs(Node<T> root){
    if (root == null) return;
    // Complete the requirement
    // ...
    dfs(root.left);
    dfs(root.right);
}
T dfs(Node<T> root){
    if (root == null) return 0;
    return dfs(root.left) + dfs(root.right) + var; // var depends on
the question
}

```

- Iterative DFS

```

void dfs(Node<T> root){
    Stack<Node<T>> pending = new Stack<>();
    pending.push(root);
    while (!pending.isEmpty()){
        Node p = pending.pop();
        // Do something...
        pending.add(root.left);
        pending.add(root.right);
    }
}
T dfs(Node<T> root){
    Stack<Node<T>> pending = new Stack<>();
    T ans /* = ... */; // Remember to initialize!!!
    pending.push(root);
    while (!pending.isEmpty()){
        Node p = pending.pop();

```

```

        // Do something to ans...
        pending.push(root.left);
        pending.push(root.right);
    }
    return ans;
}

```

- BFS

```

T BFS(Node<T> root){
    Queue<Node<T>> pending = new LinkedList<>(); // LinkedList!!!
    T ans /* = ... */; // Remember to initialize!!!
    pending.add(root);
    while (!pending.isEmpty()){
        Node p = pending.poll();
        // Do something to ans
        pending.add(root.left);
        pending.add(root.right);
    }
}

```

Graph

There are 3 ways to represent a graph, that use integer to represent the vertex:

- Adjacency Matrix: A `ArrayList<ArrayList<Integer>>` that `matrix.get(i).get(j) == true` if there is an edge from vertex `i` to vertex `j`. For any `i` and `j`, `matrix.get(i).get(j) == matrix.get(j).get(i)` if the graph is undirected.
- Incidence Matrix: A `ArrayList<ArrayList<Integer>>` that `matrix.get(i).get(j) == true` if the edge `j` is incident to vertex `i`.
- Adjacency List: A `ArrayList<ArrayList<Integer>>` that `list.get(i)` is a list of vertices that are adjacent to vertex `i`. For any `i` and `j`, `j` is in `list.get(i)` if and only if `i` is in `list.get(j)` if the graph is undirected.

There is also a way to represent a graph that use `class Vertex` to represent the vertex:

```

class Vertex {
    ArrayList<Vertex> neighbors;
    Vertex() { neighbors = new ArrayList<>(); }
}

```

To traverse a graph, we can use DFS or BFS. To avoid visiting the same vertex twice, we can use a `HashSet<Vertex> visited` to store the visited vertices.

How to choose recursive DFS, iterative DFS and BFS:

- If it can pass `HashSet<Vertex> visited`, choose recursive DFS.

- If it can NOT pass `HashSet<Vertex> visited`, choose iterative DFS.
- If we try to resolve a problem that is related to the shortest path, choose BFS.
- Recursive DFS

```
T dfs(Vertex v, HashSet<Vertex> visited){
    if (v == null || visited.contains(v)) return 0;
    visited.add(v);
    T ans /* = ... */; // Remember to initialize!!!
    // Do something to ans here
    for (Vertex neighbor : v.neighbors) ans += dfs(neighbor, visited);
    return ans;
}
```

- Iterative DFS

```
T dfs(Vertex v){
    Stack<Vertex> pending = new Stack<>();
    HashSet<Vertex> visited = new HashSet<>();
    T ans /* = ... */; // Remember to initialize!!!
    pending.push(v);
    while (!pending.isEmpty()){
        Vertex p = pending.pop();
        if (visited.contains(p)) continue;
        visited.add(p);
        // Do something to ans here
        for (Vertex neighbor : p.neighbors) pending.push(neighbor);
    }
    return ans;
}
```

- BFS(Shortest Path)

```
int bfs(Vertex start, Vertex target) {
    Queue<Vertex> pending = new LinkedList<>();
    Map<Vertex, Integer> dis = new HashMap<>(); // distance
    pending.add(start);
    dis.put(start, 0);

    while (!pending.isEmpty()) {
        Vertex p = pending.poll();
        if (p == target) return dis.get(p);

        for (Vertex neighbor : p.neighbors) {
            if (!dis.containsKey(neighbor)) {
                dis.put(neighbor, dis.get(p) + 1);
                pending.add(neighbor);
            }
        }
    }
}
```

```

    }
}
return -1; // not found
}

```

- Some Specific Example

These are implemented with adjacency matrix.

```

static ArrayList<Integer> reachableFrom(Graph g, int v) {
    // COMPLETE
    HashSet<Integer> visited = new HashSet<>() ;
    ArrayList<Integer> ans = new ArrayList<>() ;
    Stack<Integer> pending = new Stack<>() ;
    pending.push( v ) ;
    visited.add( v ) ;
    while ( !pending.isEmpty() ){
        int p = pending.pop() ;
        ans.add( p ) ;
        for ( int i = 0 ; i < g.size() ; i ++ ){
            if ( g.adjacent( p , i ) && !visited.contains(i) ){
                visited.add( i ) ;
                pending.add( i ) ;
            }
        }
    }

    return ans ;
}

static boolean isConnected( Graph g ) {
    return reachableFrom(g, 0).size() == g.size();
}

static boolean hasCycles(Graph g) {
    HashSet<Integer> visited = new HashSet<>() ;
    Stack<Integer> pending = new Stack<>() ;
    Stack<Integer> last = new Stack<>() ;
    for ( int i = 0 ; i < g.size() ; i ++ ) {
        if ( !visited.contains( i ) ) {
            last.push( -1 ) ;
            visited.add( i ) ;
            pending.push( i ) ;
            while ( !pending.isEmpty() ) {
                int now = pending.pop() ;
                int prev = last.pop() ;
                for ( int j = 0 ; j < g.size() ; j ++ ) {
                    if ( g.adjacent( now , j ) ) {
                        if ( !visited.contains( j ) ) {
                            pending.push( j ) ;
                            visited.add( j ) ;
                            last.push( now ) ;
                        }
                    }
                }
            }
        }
    }
}

```

```

        }
        else if ( j != prev ) return true ;
    }
}
}
}
}
return false;
}
static int distance( Graph g , int v , int w ) {
    assert reachableFrom( g , v ).contains( w ) ;
    if ( v == w ) return 0 ;
    int ans = -1 ;
    Stack<Integer> pending = new Stack<>() ;
    HashSet<Integer> visited = new HashSet<>() ;
    pending.push( v ) ;
    visited.add( v ) ;
    while ( !pending.isEmpty() ) {
        ans ++ ;
        int p = pending.pop() ;
        visited.add( p ) ;
        for ( int i = 0 ; i < g.size() ; i ++ ) {
            if ( g.adjacent( p , i ) && !visited.contains(i) ) {
                pending.push( i ) ;
                visited.add( i ) ;
            }
        }
    }
    return ans ;
}
static ArrayList<Integer> findPath(Graph g, int v, int w) {
    int n = g.size();
    HashSet<Boolean> visited = new HashSet<>();
    ArrayList<Integer> parent = new ArrayList<>();
    for (int i = 0; i < n; i++) parent.set(i, -1);
    Queue<Integer> queue = new LinkedList<>();
    visited.add(v);
    queue.add(v);
    while (!queue.isEmpty()) {
        int u = queue.poll();
        if (u == w) break;           // we found target
        for (int i = 0; i < n; i++) { // scan all neighbors
            if (g.hasEdge(u, i) && !visited.contains(i)) {
                visited.add(i);
                parent.set(i, u);
                queue.add(i);
            }
        }
    }
    if (!visited.contains(w)) return null; // if w was never visited,
    no path exists
    ArrayList<Integer> path = new ArrayList<>(); //reconstruct the
    path from w back to v
    for (int at = w; at != -1; at = parent.get(at)) path.add(at);
}

```



```

    Collections.reverse(path);    // now path is v → ... → w
    ArrayList<Integer> ans = new ArrayList<>();
    for (int i = 0; i < path.size(); i++)
    ans.add(path.get(path.size()-i-1));
    return ans;
}

```

Iterator

Provide a standard way to traverse a collection. Two core methods: `hasNext()`, `next()`. Decouples traversal logic from element processing. Same algorithm works for any collection that implements the iterator interface.

```

static int sumAllElements( RangeIterator i ){
    int sum = 0 ;
    while ( i.hasNext() ) sum += i.next;
    return sum ;
}

class RangeIterator {
    RangeIterator(int start, int end) { this.start = start; this.end = end
}

    boolean hasNext() { return this.start < this.end; }
    int next() {
        int current = this.start;
        this.start++;
        return current;
    }
    private int start, end;
}

```

Stream

An element sequence that can be infinite. Data is generated/loaded on demand, not all loaded into memory at once. Can be perfectly represented by an iterator. Example: Fibonacci infinite stream — `hasNext()` always returns `true`.

File I/O

File is a sequence of bytes on secondary storage. Persistent (survives power off).

Read

`FileInputStream` for raw bytes. `read()` returns `int` (0–255, or -1 for EOF).

Write

`FileOutputStream` for raw bytes. `write(int b)` writes one byte.

Close

Always call `close()` to release system resources.

Encoding

Mapping rules between characters and byte sequences. Must specify encoding when reading/writing text, otherwise garbled text appears. Common encodings: ASCII, UTF-8 (most common, Chinese takes 3 bytes), GBK.

Reader / Writer

- `InputStreamReader`: wraps byte stream, decodes bytes to chars by given encoding.
- `OutputStreamWriter`: wraps byte stream, encodes chars to bytes by given encoding.

Exception

Used to handle IO failures (file not found, no permission, disk full, etc.). Two ways to handle:

- `try-catch`: handle exception in current method.
- `throws IOException`: pass exception to the caller.

Serialization

Convert objects/data into byte sequence to write into file. Deserialization is the reverse.

Integer format

Represent non-negative integer as decimal digits ending with semicolon, e.g. `123;`.

- `readInt`: read chars until `;`, accumulate digits into integer.
- `writeInt`: write digits from highest to lowest, append `;` at end.

Array format

Write length first, then all elements, e.g. `3;10;20;30;`.

Buffering

Single-byte read/write triggers one system call each time — very slow for large files. Use a byte array as **buffer**, read/write a whole block at once, reduce system call count dramatically.

Buffered read

`int read(byte[] buffer)` — fills buffer, returns actual bytes read, returns -1 on EOF.

Buffered write

`write(byte[] buf, int offset, int length)` — write sub-range of buffer.

Random Access

`RandomAccessFile` allows jumping to any byte position in the file, not just sequential read/write.

Core methods

- `seek(long pos)`: jump to specified byte position.
- `getFilePointer()`: get current position.
- `length()`: total file size in bytes.
- `readByte()/readShort()/readInt()/readLong()` — read 1/2/4/8 bytes.

Endianness

- Big Endian: high byte first. Default of Java `RandomAccessFile`.
- Little Endian: low byte first. Used by BMP, WAV and many binary formats.
- Fix with `Integer.reverseBytes()` / `Short.reverseBytes()`.

Resource Management

If exception happens before `close()`, resource leaks.

try-with-resources

Automatically calls `close()` for all `AutoCloseable` resources. Safe and concise.

```
try (FileInputStream fis = new FileInputStream(filename)) {
    fis.read();
} catch (IOException e) {
    // handle error
}
```

BMP Format

Uncompressed bitmap format. Pixel maps directly to bytes.

File structure

1. **BMP Header (14 bytes)**: "BM" signature, file size, pixel data offset (usually 54).
2. **DIB Header (40 bytes)**: width, height, bit depth (24), compression (0 = none).
3. **Pixel Array**: stored bottom-to-top, left-to-right. Each pixel = BGR (3 bytes).
4. **Row Padding**: each row must be multiple of 4 bytes. Formula: `padding = (4 - (width * 3) % 4) % 4`.

Pixel processing

Java `byte` is signed. Use `& 0xFF` to convert to 0–255 unsigned value before calculation.

Common filters

- Grayscale: `gray = 0.299*R + 0.587*G + 0.114*B`, set all 3 channels to gray.
- Negative: `new = 255 - old` for each channel.
- Brightness: add delta to each channel, clamp to 0–255.
- Blur: each pixel = average of its 3×3 neighborhood.

Inheritance

extends keyword. "is-a" relationship. Java only supports **single** inheritance. All classes implicitly inherit **Object**. Subclass inherits all fields and methods from superclass. Code reuse.

Method Override

Subclass redefines a method from superclass. **@Override** annotation for compiler check. Polymorphism: method called depends on actual object type, not reference type.

Access levels

- **private**: only same class.
- **protected**: same class + all subclasses.
- **public**: everyone.

super keyword

- **super(args)**: call superclass constructor. Must be first line of subclass constructor.
- **super.method()**: call overridden method from superclass. Avoids infinite recursion.

final keyword

final method cannot be overridden. **final** class cannot be inherited.

Abstract Class

Declared with **abstract**. Cannot be instantiated with **new**. Can contain both concrete methods (shared code) and abstract methods (no body, must be implemented by subclass). A class with any abstract method must be declared abstract.

Interface

Declared with **interface**. Pure behavior contract, no fields, no method implementations. A class uses **implements** keyword. One class can implement multiple interfaces (multiple inheritance of type). Methods in interface are implicitly **public**. Implementing class must implement all methods.

Iterable / Iterator

- **Iterator<T>**: **hasNext()**, **next()**.
- **Iterable<T>**: **iterator()**. Any class implementing **Iterable** works with for-each loop.

Functional Interface

Interface with exactly one abstract method.

Lambda Expression

Shorthand to create an instance of functional interface, no need to write a full class. Example: **x -> Math.sin(x)** replaces an entire anonymous class.

map example

```
ArrayList<Double> map(UnaryFunction f, ArrayList<Double> a) {  
    ArrayList<Double> res = new ArrayList<>();  
    for (double x : a) res.add(f.apply(x));  
    return res;  
}
```

Visitor Pattern

Separates data structure traversal from element processing logic. Add new operations by adding new Visitor classes, no need to modify data structure code. Obeys Open-Closed Principle.

Visitor Interface

One `visit` method for each type of node.

```
interface TreeVisitor<T> {  
    void visitNode(T root);  
    void visitEmptyTree();  
}
```

accept method

Defined on each element class. Takes a Visitor, calls the matching visit method on itself. Bridge between data structure and visitor. Controls traversal order inside data structure.

```
abstract class Tree<T> {  
    abstract void accept(TreeVisitor<T> visitor);  
}  
class Node<T> extends Tree<T> {  
    void accept(TreeVisitor<T> visitor) {  
        visitor.visitNode(this.root);  
        this.left.accept(visitor);  
        this.right.accept(visitor);  
    }  
}  
class EmptyTree<T> extends Tree<T> {  
    void accept(TreeVisitor<T> visitor) {  
        visitor.visitEmptyTree();  
    }  
}
```

Concrete Visitor example — sum

```
class TreeAdder implements TreeVisitor<Integer> {  
    private int result = 0;  
    void visitNode(Integer root) { result += root; }  
    void visitEmptyTree() { }  
    int getResult() { return result; }  
}
```

Diamond Problem

Ambiguity when a class inherits from two classes that both override the same method from a common ancestor. Also causes duplicate member fields. Interface solves this: no fields, no default implementation (mostly), so no ambiguity. Java allows multiple interface implementation.

AST (Abstract Syntax Tree)

Tree representation of source code. Each node represents a syntax construct (number, addition, multiplication, etc.). Correctly represents operator precedence.

Interpreter pipeline

1. **Lexer / Tokenizer**: split source text into tokens (numbers, operators...).
2. **Parser**: build AST from token sequence according to grammar rules.
3. **Evaluator / Interpreter**: traverse AST recursively, compute value of each node.

Object class

Root of all Java classes. Every class inherits from `Object` directly or indirectly. Key methods (often overridden):

- `toString()`: string representation of object.
- `equals(Object o)`: check if two objects are equal by content.
- `hashCode()`: hash value of object. Default implementations are based on memory address.